CSD457 INTERNET OF THINGS · GROUP G-04

From zero: what our research project is actually about

A read-up that assumes no background. Start at page one and by the end you will understand the problem, the research, our idea, and our results.

Chapter	Question it answers
1	What is the Internet of Things, and why is it easy to attack?
2	What does network traffic look like to a computer?
3	What is an intrusion detection system?
4	How does a detector learn what 'normal' looks like?
5	Why must the detector keep learning?
6	What is poisoning, and what is the boiling-frog attack?
7	Why are small IoT devices especially at risk?
8	Why don't existing defenses solve it?
9	Our idea: Threat-Orthogonal Adaptation
10	How we tested it, and what we found (10B: without a list of attacks)
11	What we will do next
12	The five papers, a glossary, and where everything lives

CHAPTER 1

What is the Internet of Things, and why is it easy to attack?

The **Internet of Things (IoT)** means everyday objects connected to the internet: smart doorbells, security cameras, thermostats, baby monitors, smart plugs, fitness bands, factory sensors. Each one is a tiny computer with a network connection.

They are easy targets for three reasons:

- **They are weak computers.** A smart plug has a small chip, very little memory and a tiny power budget. There is no room for heavy antivirus software.
- **They are rarely updated or watched.** People install them and forget them, often with default passwords.
- **There are billions of them.** Take over enough of them and you have an army.

That army is called a **botnet**. The best-known one is **Mirai**, which in 2016 infected large numbers of cameras and routers using default passwords and used them to flood websites with traffic. Mirai is also in the dataset we use (N-BaIoT [4]).

> **In short:** IoT devices are small, forgotten and numerous, so attackers love them. They need protection that is light enough to run on the devices themselves.

CHAPTER 2

What does network traffic look like to a computer?

Devices talk by sending **packets**: small chunks of data, each with a size and a time it was sent. A doorbell might send a few packets a second to its cloud server; an infected doorbell running a flood attack sends thousands.

A detector does not read the contents of packets. It summarises the **pattern**: how big packets are, how many are sent, how regular the timing is, how many different destinations there are, measured over several time windows (the last 0.1 seconds, 1 second, 1 minute, and so on). Each packet becomes a list of numbers called **features**. The dataset we use has 115 features per packet, computed the Kitsune way [3].

You can picture every packet as a **dot in a space**: similar behaviour gives nearby dots. Normal traffic forms a cloud. Attack traffic forms a different cloud, usually far away.

> **In short:** To the detector, each packet is a point. Normal traffic is one cloud of points; attacks are another.

CHAPTER 3

What is an intrusion detection system?

An **intrusion detection system (IDS)** watches traffic and raises an alarm when something looks like an attack. There are two kinds:

Kind	How it works	Weakness
Signature-based	Keeps a list of known attack patterns, like an antivirus.	Misses new attacks that are not on the list.
Anomaly-based (ours)	Learns what normal looks like, alarms on anything too different.	Its idea of normal can go stale, or be manipulated.

Two numbers judge a detector:

- **Recall (detection rate):** of all attacks, what share did it catch? Higher is better.
- **False-positive rate (false alarms):** of all normal traffic, what share did it wrongly flag? Lower is better. Too many false alarms and people switch the detector off.

In short: We build an anomaly-based IDS that runs on the device itself, and we judge it by how many attacks it catches and how few false alarms it raises.

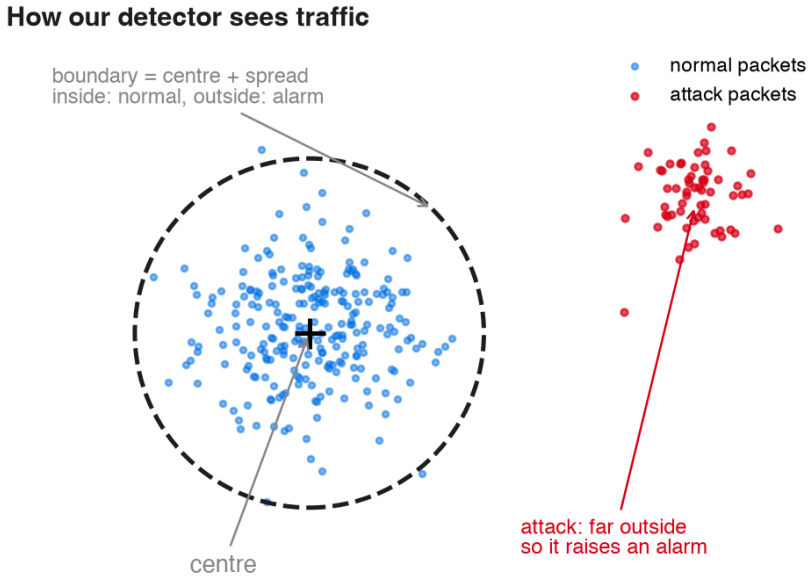
CHAPTER 4

How does a detector learn what 'normal' looks like?

Our detector keeps just two things about normal traffic:

- the **centre**: the average point of the normal cloud, and
- the **spread**: how wide the normal cloud is.

Anything within a few spreads of the centre is normal. Anything beyond that boundary raises an alarm. That's the whole detector, which is why it fits on a microcontroller.



Where do the centre and spread come from? From **learning**: the detector watches traffic it believes is normal and sets the centre to its average and the spread to its width. This is the machine learning in our project. It is simple on purpose: the attack we study exploits exactly this kind of learning [1][2].

In short: The detector is a circle drawn around normal traffic. Inside the circle means normal; outside means alarm.

CHAPTER 5

Why must the detector keep learning?

Normal is not fixed. A new device joins the network, a firmware update changes how a camera talks, people go on holiday. This slow change is called **concept drift**.

Real data shows how large it is. CESNET-TimeSeries24 [5] recorded 40 weeks of traffic from a real internet provider serving about half a million users. An analysis of it found traffic volume dropped about 52% at the end of the university semester, with significant shifts in most weeks.

A detector that learned 'normal' once and never updates will start flagging all this new normal behaviour as attacks. False alarms climb until the detector is useless. So the detector must **keep learning**: every so often it updates its centre and spread from recent traffic.

> **In short:** Normal traffic changes over time, so a useful detector has to keep updating what it thinks normal is.

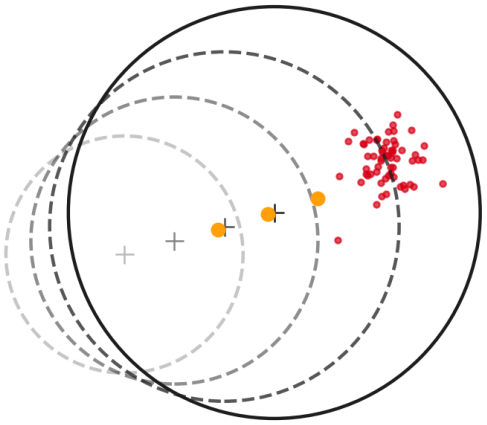
CHAPTER 6

What is poisoning, and what is the boiling-frog attack?

If the detector learns from recent traffic, an attacker can control part of what it learns from. Feeding a learning system crafted data so it learns the wrong thing is called **poisoning**.

The clever version is the **boiling-frog attack**, named after the story that a frog in slowly heating water doesn't notice until it's too late. The attacker never sends anything alarming. Instead, each cycle they send traffic placed just inside the detector's boundary, nudged toward the attack they want to hide. The detector sees it as normal and learns it. Its centre moves a little and its spread grows a little. Next cycle the attacker nudges again. Eventually the real attack sits inside the boundary and passes unnoticed.

Boiling-frog poisoning, step by step



orange = poison packets placed just inside the edge.
Each cycle the detector learns them as normal, so its centre moves and its edge grows, until the attack is inside.

This was shown on real network anomaly detectors in the paper **ANTIDOTE** [1] (2009), which also proposed a defense for large server-side systems.

> **In short:** The attacker slowly teaches the detector that attacks are normal, one tiny step at a time, without ever setting off an alarm.

CHAPTER 7

Why are small IoT devices especially at risk?

Kloft and Laskov [2] proved that how far an attacker can push this kind of detector depends on the **attacker's share** of the traffic it learns from. With a small share, honest traffic keeps pulling the detector back. With a large share, the attacker wins.

On a big network the attacker is a drop in the ocean. But a smart doorbell or plug sends very little traffic of its own, so an attacker can easily be a large share of what that device's detector sees.

Our real-data check (Chapter 10) makes this concrete: on real traffic from a smart doorbell, the naive self-learning detector was poisoned when the attacker was just **10 to 20%** of the traffic (10% for the scan attack, 20% for ack and syn).

In short: The quieter the device, the easier it is to poison. IoT devices are quiet.

CHAPTER 8

Why don't existing defenses solve it?

Approach	What it does	Why it falls short
Never learn (static)	Train once, never update.	Safe from poisoning, but goes stale under drift and drowns in false alarms.
Learn everything (naive)	Update from all traffic judged normal.	Keeps up with drift, but is poisoned (Chapters 6 and 7).
Robust statistics on a server (ANTIDOTE [1])	Uses outlier-resistant maths, retrains in batches.	Built for large server-side systems, not continuous learning on a small device.
Freeze when suspicious	Stops learning when drift looks too big.	Can be weaponized: see below.

The freeze attack (our finding)

Suppose a defense stops learning whenever drift looks suspicious. An attacker only has to push hard enough to trip the freeze, then leave. The detector is now frozen, but normal traffic keeps drifting. False alarms climb and climb until people switch the detector off. The attacker has turned the defense itself into the attack. We found no prior paper naming this, and it broke our own first design in testing.

In short: Never learning fails, always learning gets poisoned, and freezing can be abused. We need a detector that keeps learning but can't be steered.

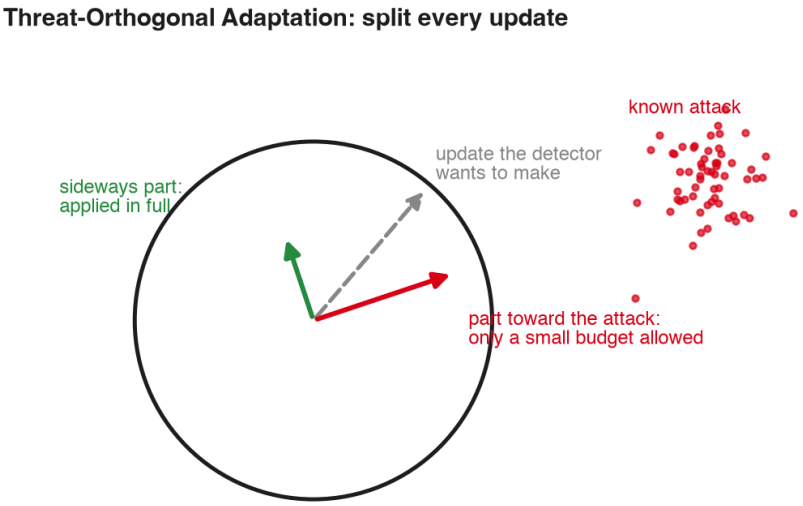
CHAPTER 9

Our idea: Threat-Orthogonal Adaptation (TOA)

The key observation: **honest drift can move in any direction, but poisoning must move toward the attack it wants to hide.** A semester-end traffic drop, for example, moves normal away from flood attacks. An attacker hiding a flood has to drag normal toward floods.

So TOA looks at every update the detector wants to make and splits it into two parts:

- **The sideways or away part** (green below): applied in full, straight away. The detector always keeps up with honest drift and never freezes, so the freeze attack doesn't work on it.
- **The part toward a known attack** (red below): allowed only up to a small, slow budget learned from clean traffic. The attacker's only useful direction is rationed.



'Orthogonal' is the maths word for 'at right angles to'. The safe part is the part at right angles to the threat direction, hence the name.

The guarantee

Because the gap between normal and a known attack can shrink by at most a fixed budget per cycle, we can compute a **minimum number of cycles** before that attack could be hidden, however clever the attacker is. In our simulation that minimum was 678 cycles, while the naive detector fell in 24.

Why it fits on an ESP32

Splitting an update needs one dot product and one distance per known attack. That's a handful of multiplications, tiny even for a microcontroller.

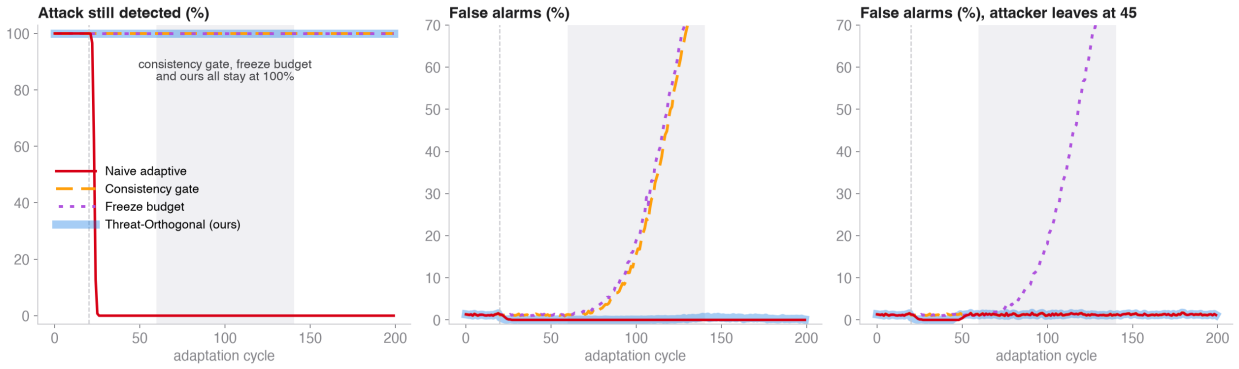
In short: Let normal move anywhere except toward known attacks. Honest change is learned freely; the attacker's only useful move is rationed.

CHAPTER 10

How we tested it, and what we found

Test 1: simulation (synthetic data)

We first built a simulated world with drift and a boiling-frog attacker controlling 40% of traffic, and compared five detectors over 5 random seeds.

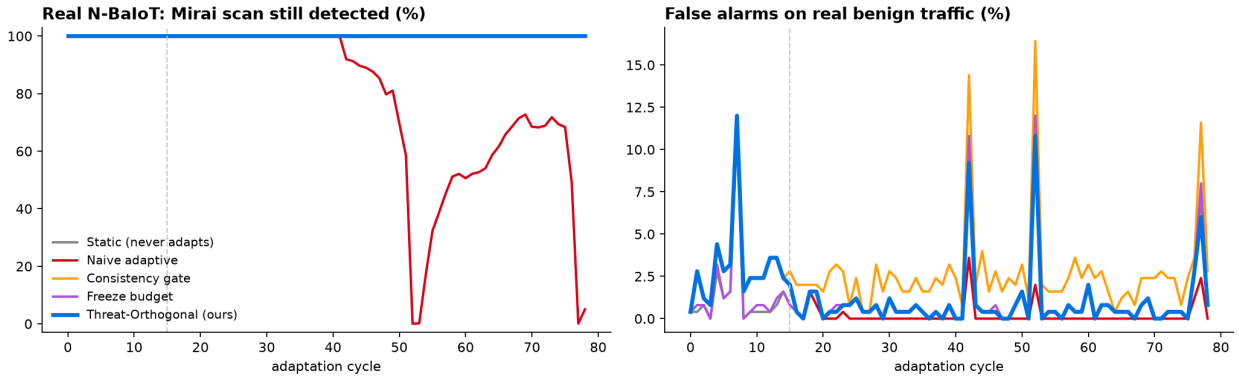


Detector	Attack still detected	False alarms
Naive learning	0% after 24 cycles	0%
Consistency filter	100%	85%
Freeze budget	100%	87%, never recovers after the attacker leaves
TOA (ours)	100%	0.3%

Test 2: real IoT traffic (N-BaloT)

Then we used **real data** from N-BaloT [4]: 49,548 packets of normal traffic from a real Danmini smart doorbell, and real Mirai attack traffic from the same doorbell. The only simulated part is the poisoner, because no dataset contains poisoning: each poison packet is a mix of a real normal packet and a real attack packet, placed just inside the boundary.

Real benign and Mirai traffic (N-BaloT, Danmini doorbell), modelled poisoner at 10% share, 5 seeds



Result on real doorbell traffic	What it means
Naive learning was poisoned with the attacker at only 10 to 20% of traffic (Mirai scan detection fell to about 5% at 10%; ack and syn survived 10% and fell at 20%).	Real IoT traffic is tightly clustered, so it is even easier to poison than our simulation suggested.

Result on real doorbell traffic	What it means
TOA kept 100% detection at every attacker share from 10% to 50%, for three different Mirai attacks (scan, ack, syn).	The defense holds on real traffic, not just synthetic.
TOA's false alarms stayed about 0.8%, the same as a detector that never learns.	The protection costs essentially nothing in false alarms.

An honest limit we found

We also let a second real device (an Ecobee thermostat) join the network midway. Every detector, even with no attacker at all, then showed about 34% false alarms. The reason: a detector that only learns from traffic it already accepts can never learn a completely different device, whose traffic always looks foreign. This is not a TOA problem but a detector-design lesson: keep **one detector per device**, which is also how N-BaIoT's authors built theirs [4].

In short: In simulation and on real IoT traffic, TOA keeps attacks visible while naive learning is poisoned, at almost no false-alarm cost. Next we must handle new devices properly and move to the ESP32.

CHAPTER 10B

Can it work without a list of attacks?

The version above needs a **signature** for each attack it protects: one reference point, such as the average of Mirai scan traffic. A fair question is whether that is just hard-coding. So we built a version that needs no signatures at all. It relies on a footprint that poison cannot avoid:

- **Learn from the dense middle only.** Poison has to sit near the boundary to have any pull without raising alarms. If the detector learns only from the dense core of traffic, that poison is ignored.
- **Accept a shift only if the whole cloud moves.** Honest change moves all of the traffic, including the outer ring. Poison moves only one part. A shift of the centre that the outer ring does not back up is scaled down.
- **Widen the boundary only if the outer ring really gets busier.** A smart attacker who hides poison inside the core tries to inflate the spread instead; this check blocks that.

Test	Signature-free result
Real doorbell traffic, normal attacker, 10 to 50% share	100% detection of Mirai scan, ack and syn; about 1% false alarms
Real doorbell traffic, smart attacker who knows the defense and hides poison in the core	100% detection; 0.2 to 0.3% false alarms
Synthetic: strong honest drift and the attack at the same time	Still poisoned in three of four settings (2 to 46% detection). Open problem

In short: The defense can work without a list of attacks, and it beat both attackers on real IoT traffic. The hard case left is an attacker hiding inside genuine, large drift. Our plan: signature-free by default, signatures as an extra safety floor for the most important attacks.

CHAPTER 11

What we will do next

When	Work
Weeks 1 to 3	Per-device detectors on N-BaIoT; more devices and attacks; attacker share taken from real device traffic rates.
Week 4 (mid-review)	Full real-data comparison with all baselines, including ANTIDOTE's robust PCA.
Weeks 5 to 7	Port the detector and TOA to the ESP32; measure memory, latency and energy. Study the freeze attack with real long-term drift from CESNET-TimeSeries24 [5].
Weeks 8 to 9	Parameter sweeps, the IEEE-format report, code and a live demo.

Open questions we will be honest about

- **Attacks hidden inside strong drift:** the signature-free version is still poisoned when heavy honest drift and the attack happen together (Chapter 10B). This is the main open research question.
- **Honest drift toward an attack:** TOA then slows learning in that direction; in simulation false alarms peaked near 7% and recovered.
- **Realism of the attacker:** our poison mixes real normal and real attack packets; a live attacker on real hardware is a stretch goal.

CHAPTER 12

The five papers, a glossary, and where everything lives

The five papers, in reading order for a beginner

[4] N-BaIoT (IEEE Pervasive Computing, 2018)
Real traffic from 9 infected IoT devices. Read first: it shows what IoT attacks look like.
https://arxiv.org/abs/1805.03409

[3] Kitsune (NDSS, 2018)
How to turn packets into features and detect anomalies on small devices.
https://arxiv.org/abs/1802.09089

[1] ANTIDOTE (ACM IMC, 2009)
The boiling-frog attack on network detectors, and a server-side defense.
https://people.eecs.berkeley.edu/~tygar/papers/SML/IMC.2009.pdf

[2] Kloft and Laskov (JMLR, 2012)
The maths of how far an attacker can push a learning detector.
https://jmlr.csail.mit.edu/papers/volume13/kloft12b/kloft12b.pdf

[5] CESNET-TimeSeries24 (Scientific Data, 2025)
40 weeks of real traffic showing how normal drifts.
https://www.nature.com/articles/s41597-025-04603-x

Glossary

Term	Meaning
Anomaly detection	Learning what normal looks like and flagging what is too different.
Botnet / Mirai	An army of hijacked devices; Mirai is a famous IoT botnet.
Concept drift	Normal behaviour slowly changing over time.
Poisoning	Feeding a learning system crafted data so it learns the wrong thing.
Boiling-frog attack	Poisoning in tiny steps that never trigger an alarm.
Attacker's share	The fraction of the learning data the attacker controls.
Freeze attack	Tripping a defense that stops learning, so drift makes it useless (our finding).
TOA	Threat-Orthogonal Adaptation: learn freely except toward known attacks (our method).
Recall	Share of attacks caught.
False-positive rate	Share of normal traffic wrongly flagged.
ESP32	A cheap, popular microcontroller with Wi-Fi, used in many IoT devices.

Where everything lives

- Repository: https://github.com/IOT-snu/brainstorming
- **topic-proposal.md**: the formal proposal.

- **notes/Presentation_and_Study_Notes.pdf**: slide script and Q&A; preparation.
- **experiments/sim_toa.py**: the simulation. **experiments/nbaiot_check.py**: the real-data check.
- **literature-review.md** and **research-findings.md**: the wider reading and how the design evolved.